CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

Practical Aim : To implement fundamental array operations.

Problem 1 :

A bakery prepares n items every morning and places them in a display row. At the end of each

hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the

front. Given the initial row and the number of hours h, print the final display order.

Describe the approach you used. How does it behave when h is very large, say 10 million? Is

there a way to get the correct result without performing the operation h times?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
rive\Desktop\code\DSA\" ; if ($?) { g++ pr11.cpp
 { .\pr11 }
Enter number of bakery items: 6
Enter bakery item IDs: 1
2
3
4
5
6
Enter number of rotation: 3
Final display order: 4 5 6 1 2 3
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

Understanding how to move the first element to the last position repeatedly was the main challenge. For a very large value of h, performing the operation one by one can take a lot of time.

Outcome/Conclusion:

The final display order was obtained successfully using left rotation. Since rotating n positions gives the original array, h % n can be used to reduce the number of rotations and make the solution efficient.

Problem 2 :

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs. Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
rive\Desktop\code\DSA\" ; if ($?) { g++ pr12.cpp
 { .\pr12 }
enter sentance :this is DSA lab
longest word :this
number of word :4
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

The main challenge was identifying which book IDs appeared more than once without missing any repeated values. Comparing every element with every other element can also become slow for a large number of records.

Outcome/Conclusion:

All book IDs borrowed more than once were identified successfully. A simple nested-loop approach works for small data, but using a frequency-counting method or hash map requires fewer comparisons and is more practical for large data such as 100,000 records.

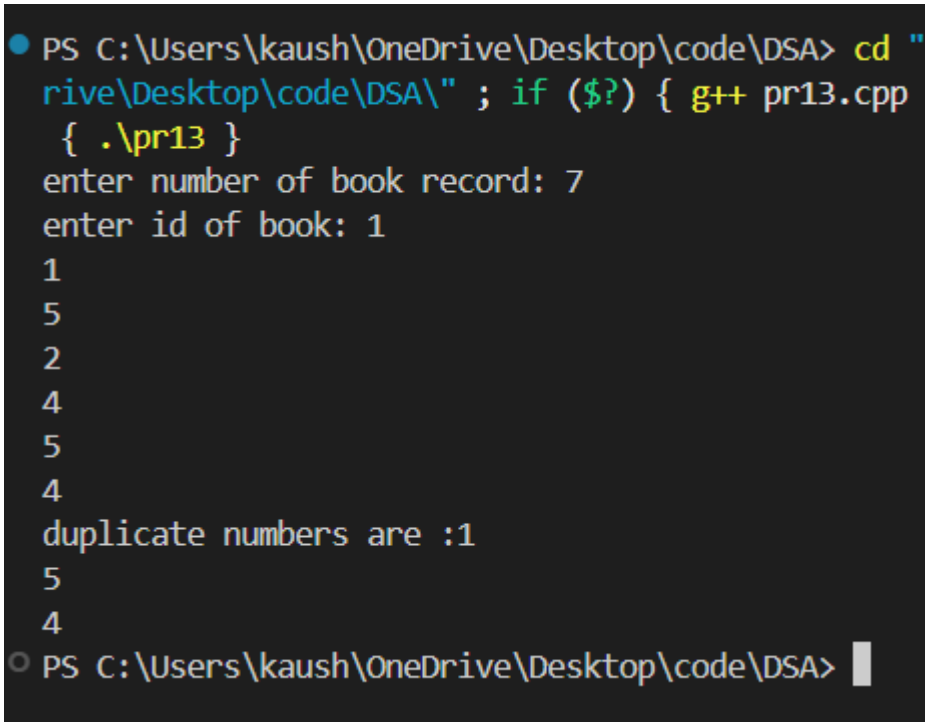
Problem 3 :

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has. Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

The main challenge was separating the sentence into words and comparing their lengths. Another case to consider is when two or more words have the same maximum length.

Outcome/Conclusion:

The longest word and its length were found successfully. The solution keeps the first longest word when two words have the same length. This behavior can be changed explicitly to print all words having the maximum length.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. **The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.**

Answer:

The display has a fixed number of n items. Each hour, all items shift one position to the left and the first item moves to the end. After exactly **n rotations**, every item returns to its original position. Therefore, the order repeats every n hours, and the first repetition occurs after n rotations.

2. **If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?**

Answer:

The book ID should appear **only once** because we only need to identify which books were borrowed more than once. The solution should avoid printing the same ID repeatedly. We can verify this by testing a log containing the same ID three times and checking that it appears only once in the output.

3. **The editor later decides that punctuation attached to a word, such as `hello,` or `great!`, should not count toward its length. How would this change your solution, and at which step would you handle it?**

Answer:

We would remove punctuation marks from each word before calculating its length. This should be handled **after splitting the sentence into words and before comparing their lengths**. Then only the actual letters of the word would be counted.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

Practical Aim :

To implement linear and binary search using both iterative and recursive approaches.

Problem 1 :

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found. Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
rive\Desktop\code\DSA\" ; if ($?) { g++ pr21Ite
rative } ; if ($?) { .\pr21Iterative }
Enter number: 20
Element found at position 2
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

ProblemFace:

The main challenge was understanding how to check each element one by one and stop when the target was found. Implementing the recursive approach was also slightly difficult because the function calls itself for the next element.

Outcome/Conclusion:

Linear search was successfully implemented using both iterative and recursive approaches. The algorithm searches from the beginning and returns the **first occurrence** of the target. Its worst-case time complexity is **O(n).**

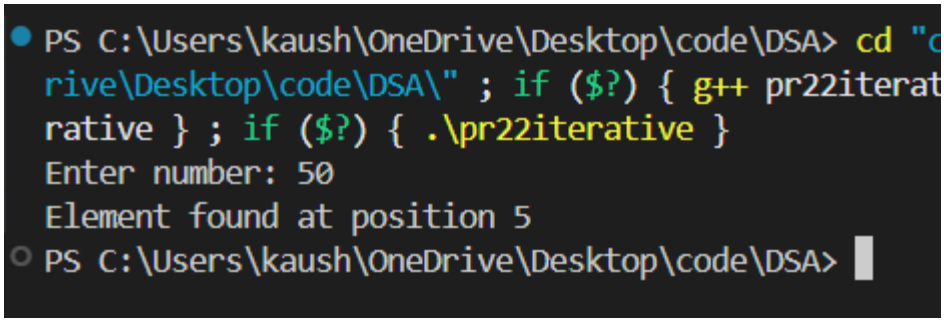
Problem 2 :

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

The main challenge was understanding how binary search uses the middle element to reduce the search area. Correctly updating low, high, and mid was important in both iterative and recursive approaches.

Outcome/Conclusion:

Binary search was successfully implemented using both iterative and recursive approaches. It finds elements efficiently in a **sorted array** with a worst-case time complexity of **O(log n)**. The array must be sorted for the algorithm to work correctly.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

ANS : The input needs to have some useful property, such as being **sorted**. If the data is sorted, we can compare the target with the middle element and decide whether to search the left or right half. Problem 2 uses this property through **binary search**, which allows us to skip many elements instead of checking every element.

2. What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

ANS: It would give a **wrong answer in some cases, but not necessarily every time**. Binary search depends on the sorted order to decide which half can contain the target. Without sorting, that decision is not reliable. It might accidentally find the target, but it can also discard the half containing the target and report that it was not found.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

Practical Aim :

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

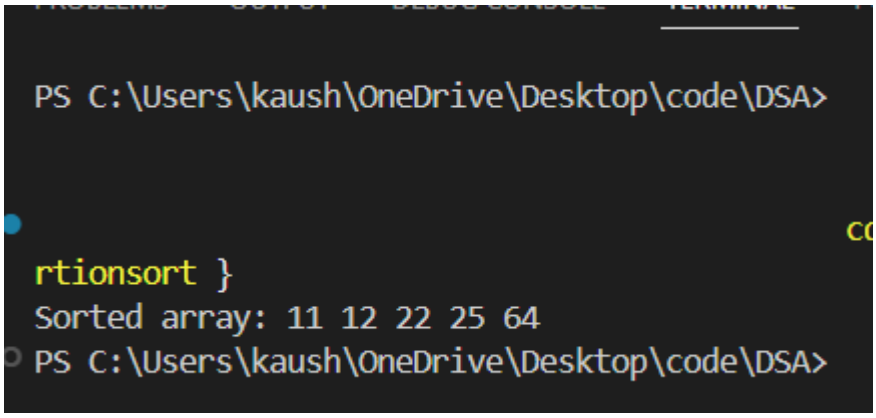
Problem 1 :

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks. Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

The main difficulty was understanding the difference between Bubble Sort, Selection Sort, and Insertion Sort because all three produce the same sorted result using different methods.

Outcome/Conclusion:

All three sorting algorithms were successfully implemented. Bubble Sort compares adjacent elements, Selection Sort selects the minimum element, and Insertion Sort inserts each element into its correct position. Insertion Sort performs efficiently when the data is already sorted.

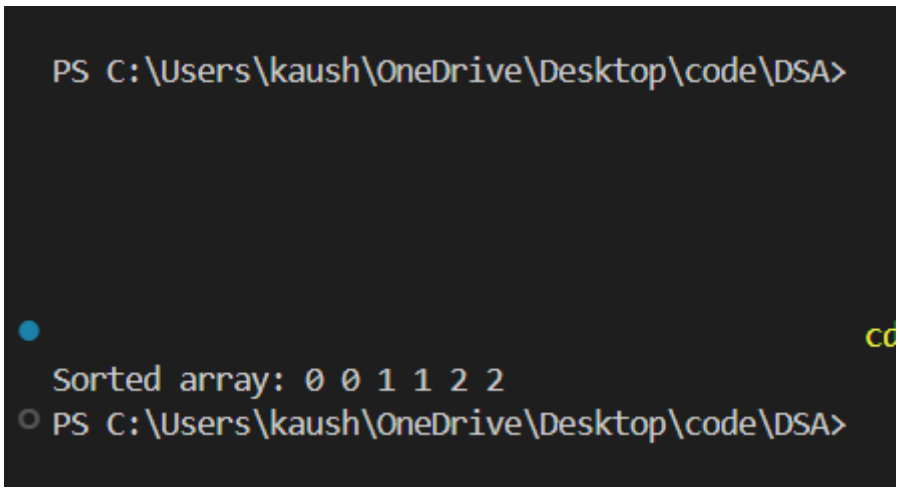
Problem 2 :

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result. Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:



Problem Faced:

The main challenge was rearranging 0, 1, and 2 without using extra storage and understanding the roles of low, mid, and high.

Outcome/Conclusion:

The array was successfully sorted in place using the three-pointer approach. The algorithm requires only one traversal, runs in **O(n)** time, and uses **O(1)** extra space.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

ANS:

Bubble Sort can detect early that the list is already sorted. We add a `swapped` variable and set it to `true` whenever a swap occurs. After a complete pass, if no swap occurred, the array is already sorted, so we stop.

The other two methods do not normally have this early-stop condition. Selection Sort still needs to find the minimum element from the unsorted part, while Insertion Sort still processes each element, even though it does very little work on an already sorted list.

2. If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

ANS :

If we first count the number of 0s, 1s, and 2s and then rewrite the array using those counts, the **rewriting step causes the second pass**.

We can eliminate the second pass by using the **three-pointer approach** with low, mid, and high. It rearranges the elements while scanning the array, so the sorting is completed in **one pass** with **O(n)** time and **O(1)** extra space.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY

AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 4: Singly Linked List: Dynamic Insertion, Deletion, and Traversal

Practical Aim :

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

Problem 1 :

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation.

Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
20
20 30
10 20 30
10 20 25 30
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

The main difficulty was understanding how nodes are connected using pointers. Inserting at different positions also required changing the next pointers correctly.

Outcome/Conclusion:

Front, end, and position insertion operations were successfully implemented using a singly linked list. Invalid positions were also handled properly.

Problem 2 :

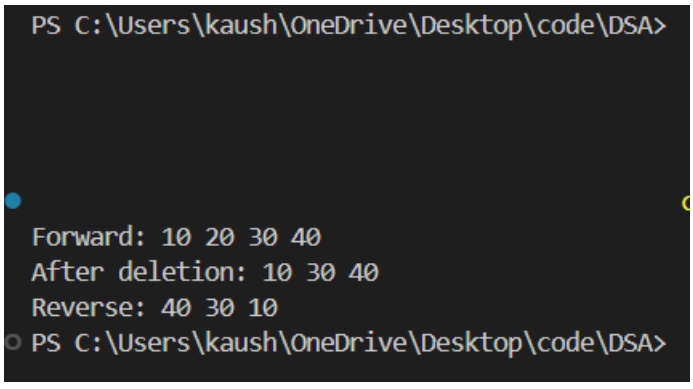
The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a. Describe how your reverse printing works. Does it use any extra storage, or does it work purely

by following the links? Can you think of another way to reverse-print without modifying the original queue?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:



Problem Faced:

The main difficulty was understanding how to delete a node without breaking the remaining links. Reverse printing was also challenging because the linked list normally moves only from front to back.

Outcome/Conclusion:

Deletion by value, forward traversal, and reverse printing were successfully implemented using a singly linked list. Reverse printing was performed using recursion without modifying the original linked list.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

ANS:
The solution should **report an error** when the position is beyond the current length of the list. Silently inserting at the end may give an unexpected result. This assumes that the caller provides a valid position and checks the error if an invalid position is given.

2. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

   ANS :

   If the node is the only node, only the **head pointer** needs to be changed. It must be set to NULL. In the middle of a list, the previous node's next pointer must be changed to point to the next node of the deleted node.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 5: Doubly Linked List and Circular Linked List Operations

Practical Aim :

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

Problem 1 :

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd '
After insert front: 20
After insert end: 20 40
After insert after 20: 20 30 40
Total songs: 3
After deleting first song: 30 40
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

The main difficulty was understanding that a doubly linked list has both prev and next pointers. While inserting or deleting a node, both links had to be updated correctly.

Outcome/Conclusion:

Insertion at the front, insertion at the end, insertion after a given song, deletion of the first song, counting, and forward traversal were successfully implemented using a doubly linked list.

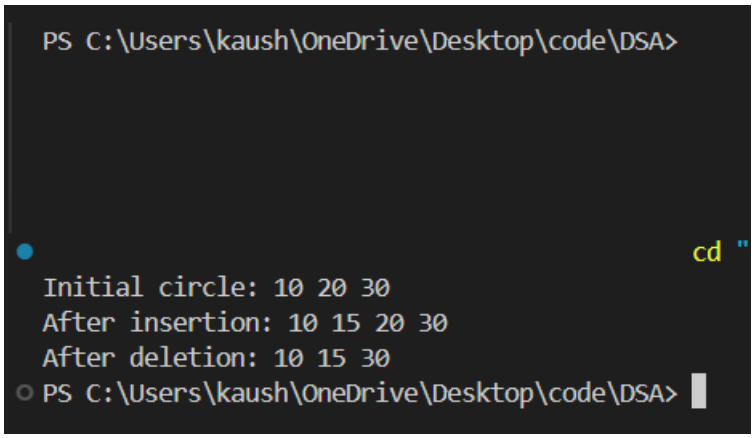
Problem 2 :

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation. Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:



Problem Faced:

The main difficulty was maintaining the circular connection after insertion and deletion. The special case of deleting the only node also required separate handling.

Outcome/Conclusion:

Singly circular and doubly circular linked lists were successfully implemented. Insertion, deletion, and traversal were performed while maintaining the circular connections between the nodes.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. A doubly linked list maintains two links per node instead of one. Whenmdeleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

ANS :

When the first node is deleted, the second node becomes the new first node. Since a doubly linked list has a prev pointer, the new first node's prev must be set to NULL. If this is not done, the new first node may still point to the deleted node, creating an invalid link and causing incorrect traversal or memory problems.

2. If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

ANS :

Traversal should stop when we reach the **starting node again**. This works because the last node points back to the first node. If the last node's link is set to NULL, the circle is broken, so the traversal cannot return to the starting node and the circular structure is no longer maintained.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 6: Stack Implementation using Array and Linked List, with Expression Processing

Practical Aim :

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

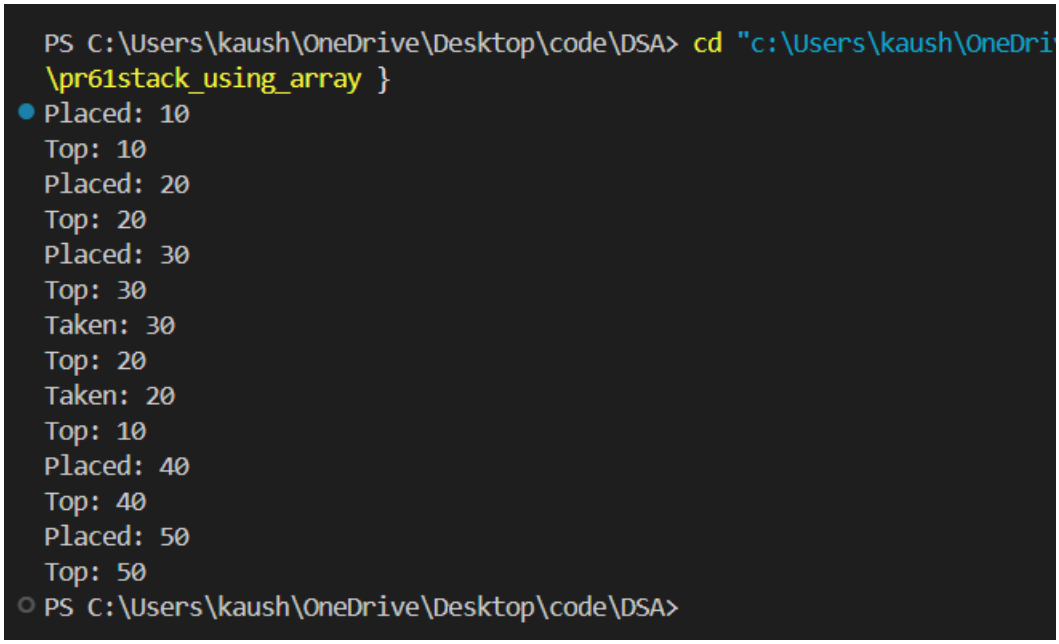
Problem 1 :

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed. Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

Understanding the top pointer and handling stack overflow and underflow conditions was the main challenge.

Outcome/Conclusion:

The stack was successfully implemented using an array. Push and pop operations were performed correctly with proper full and empty checks.

Problem 2 :

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation. Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd "c:\Users\k
?) { .\pr62stack_using_linklist }
Visited: 1
Current Page: 1
Visited: 2
Current Page: 2
Visited: 3
Current Page: 3
Back from: 3
Current Page: 2
Back from: 2
Current Page: 1
Back from: 1
No page left
No history left
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

The main difficulty was understanding dynamic node creation and how the top pointer changes during push and pop operations.

Outcome/Conclusion:

The stack was successfully implemented using a linked list. It can grow dynamically according to available memory and handles an empty stack correctly.

Problem 3 :

A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
r63infix_to_postfix }
Postfix: 342*+
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

Understanding operator precedence, brackets, and deciding when to push or pop operators from the stack was the main challenge.

Outcome/Conclusion:

The infix expression was successfully converted to postfix using a stack. Operator precedence and brackets were handled correctly.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

ANS :

In a circular array, front and rear are updated using modulo:

(index + 1) % size
This makes them wrap around to the beginning. The array is full when the next rear position becomes equal to front.

2. [Problem 2] Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

ANS : Yes, a linked-list stack can still run out of space. Its limit depends on the **available memory (RAM)** of the system. It is not fixed in the code because nodes are created dynamically.

3. [Problem 3] operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

ANS : If all operators have equal precedence, operators may be placed in the **wrong order** in postfix form. This can produce an incorrect calculation result. * and / should have higher precedence than + and -.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

Practical Aim :

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1 :

A government office has a token counter that issues tokens to visitors. The counter can hold at

most n tokens at a time. New visitors join from one end and are served from the other end in the

order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one

can be served. Given a sequence of join and serve operations, implement this fixed-capacity

token system and print the current front token after each operation. Report an error if an

operation cannot be performed.

Describe how your solution tracks the front and rear of the queue. What happens to your front

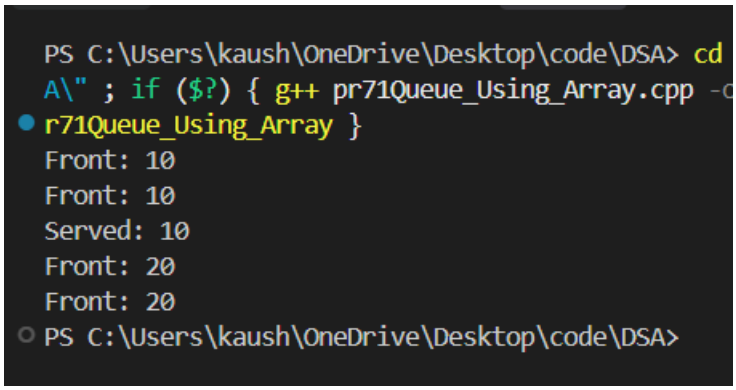
and rear pointers over time as join and serve operations keep happening, and does your

solution handle the case where space appears to run out even though slots have been freed?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

Understanding the top pointer and handling stack overflow and underflow conditions was the main challenge.

Outcome/Conclusion:

The stack was successfully implemented using an array. Push and pop operations were performed correctly with proper full and empty checks.

Problem 2 :

A hospital emergency ward receives patients continuously and attends to them strictly in the

order they arrived, with no upper limit on how many can be waiting at once. New patients are

added at the back and the doctor always attends to the patient at the front. Given a sequence of

arrive and attend operations, implement this unlimited patient queue and print the current front

patient after each operation.

Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
A\" ; if ($?) { g++ pr72Queue_Using_Linked_List.
f ($?) { .\pr72Queue_Using_Linked_List }
Front: 10
Front: 10
Front: 10
Served: 10
Front: 20
Served: 20
Front: 30
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

Managing front and rear pointers and understanding dynamic memory allocation.

Outcome/Conclusion:

Successfully implemented queue operations using arrays and linked lists.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

ANS :

In an array queue, front moves forward during deletion, but rear does not move backward. So, the queue becomes full even when front spaces are empty.

2. [Problem 2] Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

ANS :

If arrivals exceed attendances, the queue keeps growing and uses more memory. This is not visible because the code has no memory limit check.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 8: Binary Tree Traversals and Binary Search Tree Insertion

Practical Aim :

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree..

Problem 1 :

A company stores its organizational hierarchy as a tree — the CEO at the top, managers below,

and employees at the leaves. Different departments need the hierarchy printed in different

orders: HR wants each manager printed between their left and right sub-teams, the archive

department wants each manager printed before their sub-teams, the payroll department wants

each manager printed after all their sub-teams are listed, and the floor manager wants everyone

printed level by level from top to bottom. Given a binary tree representing the hierarchy,

implement all four printing orders and display the result of each.

Describe how your solution for level-by-level printing differs from the other three. What

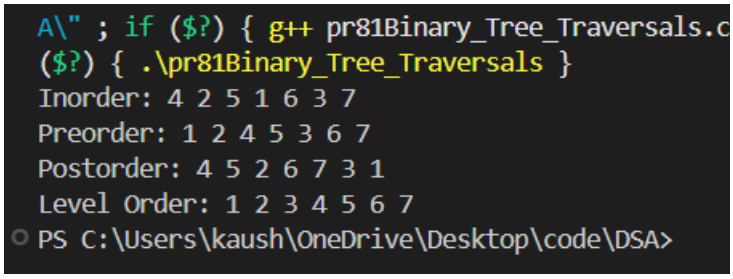
additional structure did you need to use, and why does the same recursive approach that works

for the other three not directly apply here?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

Understanding different tree traversals and implementing level-order traversal using a queue.

Outcome/Conclusion:

Successfully implemented all four tree traversals: inorder, preorder, postorder, and level-order.

Problem 2 :

A school library uses a system where every book is assigned a unique code when it arrives.

Books are organized so that at any shelf position, all books to the left have smaller codes and

all books to the right have larger codes. This arrangement lets staff find any book quickly by

always knowing which direction to look. Given a sequence of book codes arriving one by one,

insert each into the correct position and print the inorder sequence after all insertions are

complete.

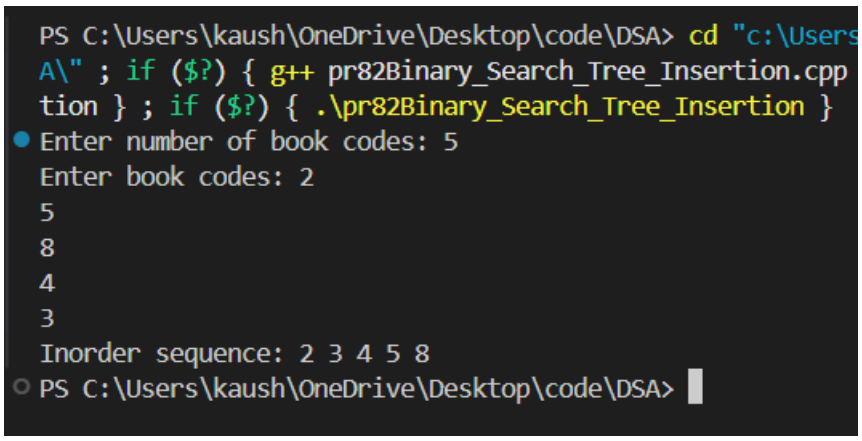
Describe how your solution decides where to place each new book code. What property of the

final inorder sequence tells you whether all insertions were performed correctly?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:



Problem Faced:

Inserting nodes at the correct position in a BST while maintaining its properties.

Outcome/Conclusion:

Successfully implemented BST insertion and verified the sorted output using inorder traversal.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

ANS :

Preorder alone cannot uniquely reconstruct a tree. Inorder traversal or additional structural information is needed.

2. [Problem 2] The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both? How would you isolate which one?

ANS :

An unsorted inorder output indicates an error in insertion or traversal logic. Check the BST structure and verify the inorder traversal separately.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 9: Graph Traversal: Depth-First Search and Breadth-First Search

Practical Aim :

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree..

Problem 1 :

A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is

mapped as a network where buildings are connected by roads. One team starts at a building

and goes as deep into each road as possible before backtracking, while a second team spreads

out level by level — first checking all buildings directly connected to the start, then all buildings

two roads away, and so on. Given the city map as a graph and a starting building, implement

both exploration strategies and print the order in which buildings are visited by each team.

Describe how the order of buildings visited differs between the two strategies on the same

graph. What data structure does each strategy rely on to decide which building to visit next, and

how does that structure cause the difference in order?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd "c:\Users\kaush\OneDrive\Desktop\code\DSA\" ; 
d_BFS }
Enter number of nodes and edges: 6 6
Enter edges:
0 1
0 2
1 3
1 4
2 5
4 5
Enter starting node: 0
DFS: 0 1 3 4 5 2
BFS: 0 1 2 3 4 5
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Outcome/Conclusion:

Learned to implement DFS using recursion and BFS using a queue to traverse a graph.

Problem: Faced:

Understanding the difference between DFS and BFS and managing visited nodes correctly.

Key Questions / Analysis / Interpretation

1. Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.

Answer:

BFS finds a nearby building sooner because it explores level by level. DFS may find a deep building sooner if it follows the correct path.

2. Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure?

Answer:

Recursive DFS may cause stack overflow on a very large graph. Using an iterative DFS with an explicit stack can avoid this problem.

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 10: Hash Tables: Linear Probing, Chaining, and Double Hashing

Practical Aim :

To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

Problem 1 :

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its

registration number is used to assign it a slot by taking the last digit of the number. If that slot is

already taken, the vehicle moves to the next available slot in order, wrapping around if

necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and

display the final state of all ten slots.

Describe what your solution does when it encounters an already occupied slot. What happens

when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you

think of a situation where your solution might loop indefinitely?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush> cd "c:\Users\kaush\OneDrive\Desktop\code\DSA\" ; if ($?) { g++
Probing.cpp -o pr101Linear_Probing } ; if ($?) { .\pr101Linear_Probing }
Enter number of vehicles (max 10): 3
Enter vehicle registration numbers:
12
25
49

Parking Slots:
0 : Empty
1 : Empty
2 : 12
3 : Empty
4 : Empty
5 : 25
6 : Empty
7 : Empty
8 : Empty
9 : 49
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Outcome/Conclusion:

Learned to implement a hash table using linear probing to handle collisions.

Problem: Faced:

Handling collisions and finding the next available slot in the hash table.

Problem 2 :

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide

which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can

hold multiple books stacked together — when two books map to the same shelf, they are simply

added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf

and display the final contents of all shelves.

Describe how your solution handles two books that map to the same shelf. What happens to

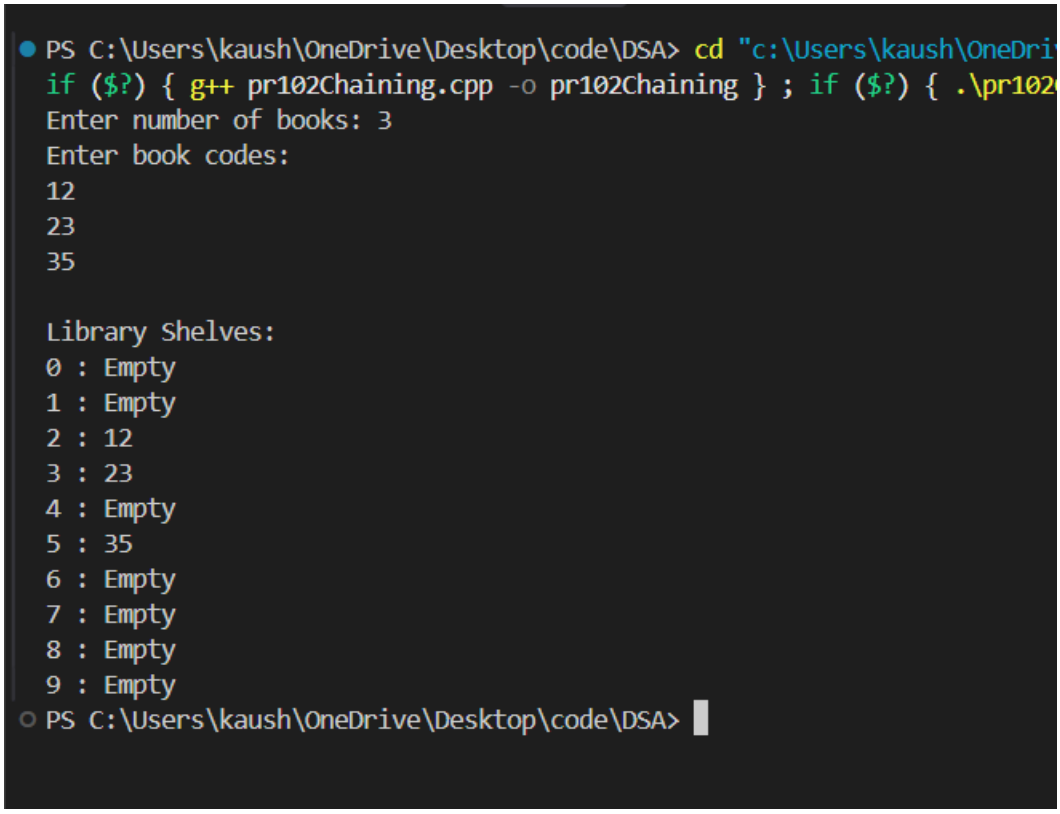
search time on a shelf as more and more books pile onto it? Can you think of what would cause

all books to end up on the same shelf?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Outcome/Conclusion:

Learned to implement a hash table using chaining to handle collisions.

ProblemFaced:

Storing multiple book codes at the same shelf and managing collisions.

Problem 3 :

A university assigns student IDs to storage slots using a two-step rule. The first rule determines

the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule

based on the ID itself determines how many slots to jump. This means different IDs that collide

will probe different sequences of slots, spreading them more evenly. Given a sequence of

student IDs, assign each to a slot using both rules and display the final state of the table.

Describe how your solution decides how many slots to jump when a collision occurs. What is

the risk if the jump size happens to be a factor of the table size? How does this strategy differ

from Problem 10.1 in how it spreads collisions?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
                                                cd "c:\Users\kaush
if ($?) { g++ pr103Double_Hashing.cpp -o pr103Double_Hashing }
ng }
Enter number of student IDs (max 10): 3
Enter student IDs:
23
1
2

Hash Table:
0 : Empty
1 : 1
2 : 2
3 : 23
4 : Empty
5 : Empty
6 : Empty
7 : Empty
8 : Empty
9 : Empty
```

Outcome/Conclusion:

Learned to implement double hashing to resolve collisions using two hash functions.

Problem Faced:

Calculating the jump size and finding an empty slot when collisions occur.

Key Questions / Analysis / Interpretation

Q1. If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy.

Answer: The first vehicle occupies its hashed slot, and the remaining vehicles occupy consecutive slots. This causes clustering and makes future insertions slower.

Q2. Suppose all book codes are multiples of 10. Predict the final shelf layout and explain why.

Answer: All books will be stored on shelf 0 because every multiple of 10 has a remainder of 0 when divided by 10. This increases search time on that shelf.

Q3. What would happen if the second hash function always produced a jump size of zero? Explain why.

Answer: The algorithm repeatedly checks the same slot and cannot resolve collisions. Therefore, the second hash function must produce a suitable non-zero jump size.